VIVA NOTES

Predictive Thermal Battery Management System for Li-ion Battery Packs

The whole project in simple words, likely questions with answers, and a glossary

Muskan Sulathana · Guide: Dr. M. Rupesh

How to use these notes: read sections 1 to 12 once, slowly. Then read the questions in section 13 and answer each one aloud in your own words before you read the model answer. Every technical word is explained in the glossary at the end. All results are simulation results: the hardware is not built yet.

Contents

1. The problem
2. The idea
3. The system: two layers
4. The components
5. How each measurement works
6. The heat model of a cell
7. The Kalman filter in plain words
8. Abnormal heat and the risk levels
9. Forecasts and the 5-minute check
10. The leak estimate
11. The tools
12. The results so far
13. Likely viva questions
14. Glossary

1. The problem

A lithium-ion cell stores a lot of energy in a small can. It is safe inside its limits. Outside them it can be damaged, and it can become dangerous.

- **Heat in a healthy cell.** Current flows through the cell's small internal resistance R. This makes heat, I^2R, called Joule heat. This heat is normal and expected.
- **Heat from a fault.** A defect or an internal short makes extra heat. The current does not explain this heat.
- **Thermal runaway.** If a cell gets hot enough, reactions inside it start to make heat. More heat makes the reactions faster, which makes more heat. This chain can end in fire. Disconnecting the pack does not stop heat that is made inside the cell.

Why a fixed limit is late. A normal BMS waits until the cell surface reaches a fixed temperature, for example 55 °C. But a cell is a large body. It warms slowly. With a fault inside, the surface may need many minutes, or may never, reach 55 °C. By then the fault has been there for a long time.

2. The idea: predict, compare, flag

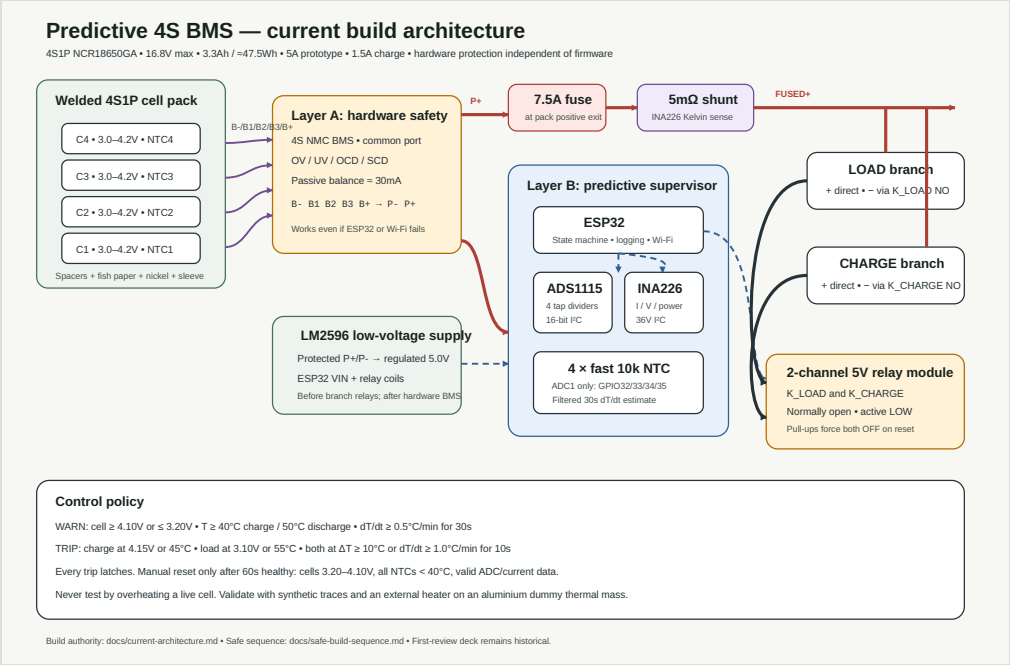
The project uses a **model** of how a healthy cell heats. Five times a second the supervisor:

1. **Predicts** the cell's temperature from its current, using the model;
2. **Compares** the prediction with the real reading of the temperature sensor;
3. **Flags** the difference. Heat that the current cannot explain is called **abnormal heat**, in watts.

A healthy cell under load has abnormal heat near zero, because its current explains its heating. So normal use is never flagged. A fault shows positive abnormal heat within about a minute, while the cell is still cool.

This follows the predict–compare–flag structure of the base paper, Ojo et al. (IEEE Transactions on Industrial Electronics, 2021). The base paper uses a neural network. This project uses a physical heat model and a Kalman filter instead. It needs no training data, every output has a unit (°C or W), and it runs on a low-cost ESP32.

3. The system: two layers



The architecture (revision 2.2): power, protection and sensing.

The pack. Four NCR18650GA cells in series (4S1P): 14.4 V nominal, 16.8 V full, 3.3 Ah, about 47.5 Wh. The prototype is limited to 5 A discharge and 1.5 A charge.

Layer A, the hardware BMS. A ready-made 4S protection board. It always works, even if the ESP32 fails. It stops charging at about 4.25 V per cell, stops discharging at about 2.7 V per cell, protects against short circuits, and balances the cells (30 mA from 4.18 V).

Layer B, the ESP32 supervisor. It measures every cell voltage, the pack current, four cell temperatures and the air temperature, five times a second. It runs the predictive layer and drives two relays:

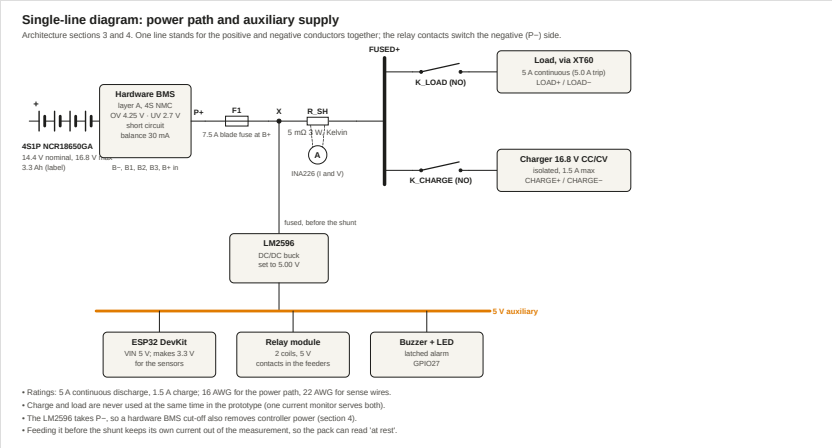
- **K_LOAD** connects the load, and **K_CHARGE** connects the charger. They are separate, so the supervisor can stop charging but allow discharge, or the other way round.
- Both are **normally open**, with pull-up resistors on their inputs. If the ESP32 resets, crashes or loses power, both relays open by themselves.
- A trip is **latched**: it stays until a deliberate manual reset, and the reset is allowed only when everything is safe again.

Why two layers? So that no single failure removes all protection. The hardware BMS is the final backstop, and nothing in the program tries to override it.

4. The components and why each one is there

Part	Why it is there
4 × NCR18650GA cells (4S1P)	The battery: 3.3 Ah each, matched at Gate 4.
4S hardware BMS (20 A board)	Layer A: voltage, short-circuit protection and balancing that always works.
ESP32 DevKit	Layer B: the supervisor. Runs the predictive layer and the dashboard.
ADS1115 (16-bit ADC)	Measures the four cell taps accurately.
330 kΩ / 56 kΩ dividers, 10 kΩ + 100 nF filters, clamps	Bring the tap voltages down to the ADC's range, remove noise, protect the inputs.
5 × 10k B3950 NTC thermistors	Four on the cells, one in the air.
INA226 module + 5 mΩ shunt	Measures the pack current in both directions.
LM2596 buck converter (5.0 V)	Powers the ESP32 and the relay module from the pack.
Dual 5 V relay module	The two disconnects, K_LOAD and K_CHARGE.
7.5 A blade fuse and holder	Protects the wiring against a large fault current.
Buzzer and red LED	A local alarm that works without Wi-Fi.
XT60 and JST-XH connectors, wire	Power, charger and balance connections.
Nickel strip, spacers, fish paper, sleeve	Build the welded pack safely.
Two perfboards (no PCB)	Board A: the cell taps and the ADS1115. Board B: the ESP32, NTC dividers, INA226, relay inputs and alarm.
PETG enclosure	A vented case. It is not a fire enclosure.
16.8 V, 1.5 A CC/CV charger	Charges the pack through the charge relay.

The full parts list costs about ₹9,260 at replacement prices (Build Guide, section 6).



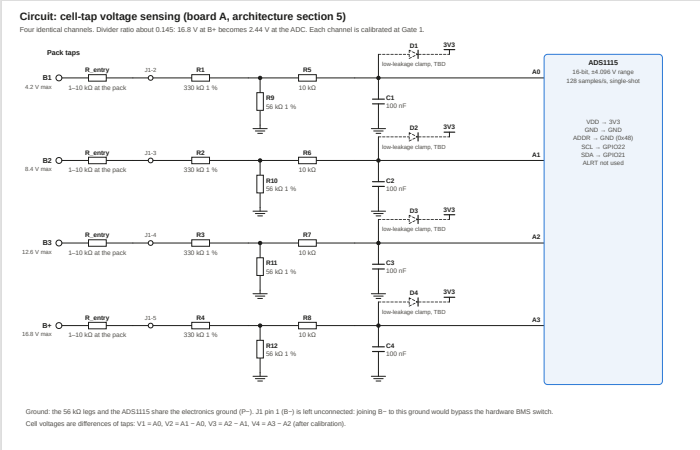
Single-line diagram: cells, BMS, fuse, shunt, relays, load and charger.

5. How each measurement works

Cell voltages: dividers and the ADS1115

The ADS1115 measures four "taps" of the pack against its negative end: 4.2, 8.4, 12.6 and 16.8 V when full. Each tap goes through a 330 kΩ and 56 kΩ divider, so 16.8 V becomes about 2.4 V. Then a 10 kΩ resistor and a 100 nF capacitor filter out noise. Cell 1 is the first tap; every other cell is the difference of two neighbouring taps.

KiCad simulation: at full charge the ADC inputs are 0.59, 1.19, 1.78, 2.38 V, all below the 3.0 V limit. Every 10 nA of leakage at an ADC input moves a cell reading by about 4.1 mV, so the clamp must leak far less than 49 nA. The filter's cut-off is about 27 Hz.



One cell-tap channel.

Temperatures: NTC dividers

An NTC thermistor's resistance falls as it gets hotter (10 kΩ at 25 °C). With a 10 kΩ resistor from 3.3 V above it, the voltage in the middle tells the temperature. The ESP32's ADC1 reads it.

KiCad simulation: 1.65 V at 25 °C, about 1.00 V at 45 °C and 0.76 V at 55 °C. The reading stays in the ESP32's accurate range from about 3 °C to above 100 °C.

Current: shunt and INA226

A 5 mΩ resistor (the shunt) sits in the pack's current path. The current makes a small voltage across it: V = I × R. The INA226 reads this voltage in both directions. Discharge is positive, charge is negative.

KiCad simulation: 25.0 mV at 5 A, well inside the INA226's ±81.9 mV range. If the module's own 0.1 Ω shunt (R100) is left in place, the reading is about 4.8 % low. So it is removed.

6. The heat model of a cell

Each cell has two parts in the model (from Lin et al., IEEE TCST 2013):

- the **core**, where heat is made. Its heat capacity is C_core.
- the **surface** (the can), where the NTC sits. Its heat capacity is C_surf.

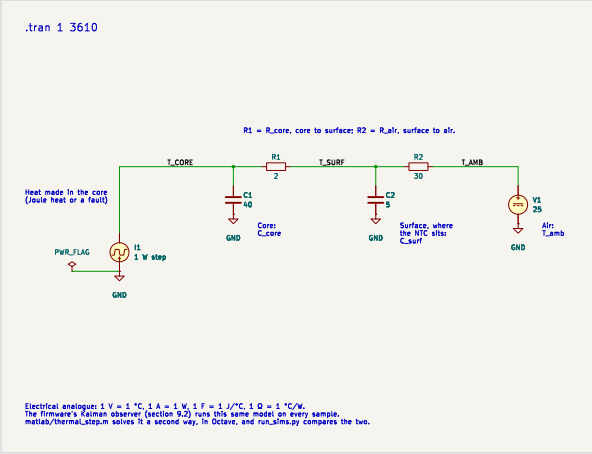
Heat flows from the core to the surface through a thermal resistance R_core, and from the surface to the air through a thermal resistance R_air. A **heat capacity** (J/°C) says how much heat warms a body by 1 °C. A **thermal resistance** (°C/W) says how many degrees it takes to push 1 W through a path.

The values used now are development values: C_core 40 J/°C and C_surf 5 J/°C (45 J/°C together, as the architecture says for an 18650), R_core 2 °C/W and R_air 30 °C/W. Gate 9 replaces them with values fitted on the real cells.

The electrical analogue

Heat flow behaves like electric current. So the model can be drawn as a circuit:

Heat	Electric circuit
Temperature, 1 °C	Voltage, 1 V
Heat flow, 1 W	Current, 1 A
Heat capacity, 1 J/°C	Capacitor, 1 F
Thermal resistance, 1 °C/W	Resistor, 1 Ω
Air temperature	A fixed voltage source
Heat made in the core	A current source



The heat model drawn in KiCad.

Result (KiCad, Octave and Xcos give the same): with 1 W in the core and 25 °C air, the core reaches about 26.3 °C after 1 minute, 31.1 °C after 5 minutes, 43.3 °C after 20 minutes and 54.5 °C after one hour. It would settle at 57 °C (25 + 1 W × 32 °C/W). There are two time constants: 8.4 s (core and surface share heat) and 1422 s, about 24 minutes (the whole cell warms up to the air). That slow time constant is why a fixed limit acts late.

7. The Kalman filter in plain words

The supervisor runs one Kalman filter per cell, five times a second. Its "state" is three numbers: the core temperature, the surface temperature, and the abnormal heat.

1. **Predict.** From the last state and the measured current, the model predicts the new state. The expected heat is I^2R, from the cell's measured resistance.
2. **Compare.** It compares the predicted surface temperature with the NTC reading.
3. **Correct.** It moves the state towards the reading. How much depends on how much it trusts the model and the sensor. If the reading stays above the prediction, the only way to explain it is more heat, so the abnormal-heat estimate rises.

So the filter gives two things no sensor measures: the **core temperature** and the **abnormal heat in watts**. A simple everyday picture: you know how fast a kettle should warm on a given flame. If it warms faster, something else is adding heat, and you can say how much.

Safety details: the current used in the calculation is limited to 5 A, and the resistance to its expected range, so a bad reading cannot "explain away" a fault. The model's values are fitted once at Gate 9 and then fixed, so the model can never slowly learn a fault as normal.

8. Abnormal heat and the risk levels

Abnormal heat is shown in watts and also as the heating rate it causes: watts ÷ 45 J/°C × 60 gives °C per minute. So 0.4 W is about 0.5 °C/min, and 0.75 W is about 1.0 °C/min.

Level	When	Action
0 Normal	None of the conditions below	—
1 Watch	5-minute check +1.0 °C or more; a cell differs from the others by 0.3 °C/min for 30 s; time to limit under 10 minutes, or current above the safe current for 30 s; leak estimate 20 mA or more; resistance drift over 30 %	Notice on the dashboard and in the log
2 Warning	Abnormal heat 0.5 °C/min (about 0.4 W) for 30 s; or a thermal Watch and an electrical Watch on the same cell together	Alarm on; both relays stay on
3 Trip	Abnormal heat 1.0 °C/min (about 0.75 W) for 10 s; a cell differs from the others by 0.6 °C/min for 10 s; self-acceleration while at Warning	Both relays open, latched and logged

A small fault raises the alarm only when two independent kinds of evidence, thermal and electrical, agree. A clear fault trips on its own. The rules are fixed tables, so every level can be traced back to the measurement that caused it.

Other fixed limits (section 8 of the architecture)

Condition	Warning	Trip
Any cell high	4.10 V	4.15 V: charge off
Any cell low	3.20 V	3.10 V: load off
Charge temperature	40 °C	45 °C: charge off
Discharge temperature	50 °C	55 °C: load off
Temperature spread	7 °C	10 °C: both off
Pack current	4.5 A	5.0 A: load off; 1.6 A: charge off

9. Forecasts and the 5-minute check

- **Predict and confirm.** Every 10 s the supervisor predicts each cell's surface temperature 1 and 5 minutes ahead. When that time comes, it checks the prediction against the real reading. Within ±0.5 °C is "confirmed normal". +1.0 °C or more raises Watch. Example: a small 0.2 W fault is only about 0.27 °C/min, below the warning level, but after 5 minutes the cell is about 1.3 °C hotter than predicted, so it raises Watch.
- **Accuracy record.** It keeps the RMS error of its predictions. Gates 9 and 10 require 0.5 °C RMS or less at 5 minutes. In the simulation the error in normal use was 0.11 to 0.12 °C.
- **Settling temperature and time to limit.** From the current, the model says where each cell will settle, and how many minutes until it reaches its limit (45 °C when charging, 55 °C when discharging).
- **Safe continuous current.** The largest current at which every cell settles below its limit. It is near 5 A at 25 °C and lower on a hot day.
- **Online resistance.** Each change in current of 0.5 A or more gives each cell's resistance, $R = \Delta V / \Delta I$. Rising resistance means ageing or a weak connection. A drift over 30 % raises Watch.
- **Self-acceleration.** At Warning, if the abnormal heat has grown by 50 % while the core rose by 1 °C within 120 s, the heat is feeding itself, as at the start of thermal runaway. The supervisor trips at once.

10. The leak estimate and the energy cross-check

An internal short drains its own cell, even at rest. So it leaves an electrical trace as well as a heat trace.

1. The supervisor waits for rest: current below 20 mA, no cell above 4.15 V, and 30 minutes after the last load or charge.
2. It compares each cell's voltage with the mean of the four. This cancels the drift that all cells share.
3. Over 60 minutes, the slope of that difference gives the leak current, using the cell's voltage-to-charge curve measured at Gate 10.

Example from the architecture: a 20 mA leak lowers a mid-charge cell by about 6 mV per hour compared with the others. That leak makes only about 0.07 W of heat, too little to see as heat. So the electrical check finds a soft short long before it is hot.

Energy cross-check. A leak current I at cell voltage V makes V × I watts of heat inside the cell. When this matches the abnormal heat from the Kalman filter, two independent methods point to the same fault.

Simulation result (Gate 11 soft short, 47 Ω across cell 2): the leak estimate read about 73 mA against about 79 mA really drawn, and the abnormal heat and the leak heat agreed: level Warning.

11. The tools, and why each one

Tool	What it does in this project	Why this tool
KiCad 10 + ngspice	The four analog circuits (cell taps, NTC divider, current shunt) and the heat model drawn as a circuit, simulated and checked	Free, open source, used in industry. ngspice is the circuit simulator inside KiCad.
GNU Octave	The heat model and the Kalman filter written a second time and compared with the controller program	Free. It runs MATLAB .m files, so the work also runs in MATLAB, which needs a paid licence.
Scilab Xcos	The heat model and the abnormal-heat detection as block diagrams	Free. It is the block-diagram tool like Simulink.
The C++ simulator	Runs the real controller code on a PC against a simulated pack: 30 test scenarios and the dashboard demonstrations	The same code that runs on the ESP32 is tested before any cell is connected (software-in-the-loop).
ESP32 + ESP-IDF	The real controller. The program is C++, on FreeRTOS, with the safety task at 5 Hz and a watchdog	ESP-IDF is Espressif's official toolset. It builds the firmware without warnings.
The dashboard	One web page: cell cards, charts, the system drawing, the demonstrations	The ESP32 serves it over its own Wi-Fi; on a PC the simulator serves it.

How the checks fit together

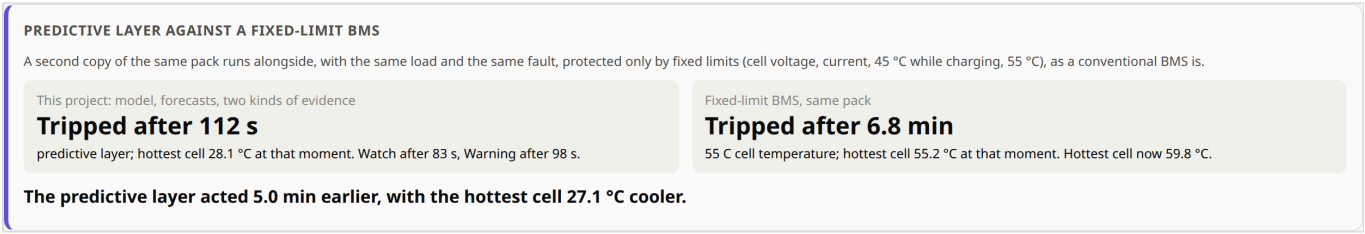
- **KiCad** checks the electronics.
- **Octave** checks that the controller program computes the model correctly.
- **Xcos** shows the method as a block diagram.
- **The C++ simulator** tests the complete program against every fault the build gates ask for.

12. The results so far (all in simulation)

The guided demonstrations against a fixed-limit BMS

Demonstration	This project	Fixed-limit BMS
Heat with no current, 1 W in cell 2	Trip about 1 minute after the heat began, cell near 26 °C	No reaction
A cell defect under 3 A	Trip about 2 minutes after the fault, near 28 °C; the cell then cools	Trip about 7 minutes after the fault, at 55 °C
Thermal runaway onset (heat in cell 3 that grows as it warms)	Trip about 8 minutes after it began, near 29 °C	Trip about 4 minutes later, at 55 °C
Soft internal short, 47 Ω (Gate 11)	Warning: leak about 73 mA against 79 mA drawn; heat and leak agree	No reaction
NTC lifted off its cell under 3 A	Sensor fault within about 10 s; both relays open	No reaction
Normal use (discharge, rest, charge)	Normal throughout; 5-minute error 0.11 to 0.12 °C RMS	—

Main result: for the cell defect, this project acts about 5 minutes earlier, with the cell about 27 °C cooler. For heat made inside a cell, no BMS can stop the heat by disconnecting; what the predictive layer gives there is warning time.



The dashboard's comparison after the cell-defect demonstration.

Tests and cross-checks

- **26 unit tests and 30 scenarios** pass. The scenarios rehearse Gates 1, 2, 9, 10 and 11 with each gate's own pass rules.
- **KiCad:** all 11 circuit checks pass; KiCad's electrical rules check finds no violation.
- **ngspice and Octave** solve the heat model two ways and agree within 0.0002 °C.
- **Octave against the controller program**, every sample replayed:

Scenario	Largest core-temperature difference	Largest abnormal-heat difference
`discharge_steps`	0.0029 °C	0.0013 W
`heat_no_current`	0.0066 °C	0.0013 W
`load_defect`	0.0026 °C	0.0013 W
`soft_short`	0.0040 °C	0.0010 W

- **Xcos:** the heat model built from blocks agrees with ngspice and Octave within 0.004 °C.
- **Xcos:** with 1 W of abnormal heat, the predictive trip comes 117 s after the fault starts (surface at 30.9 °C). A fixed 55 °C limit trips 32 minutes after it starts. The estimate settles at the true 1.00 W, and stays at zero before the fault: no false alarm.

What is not done yet

- The hardware is not built. It is built gate by gate, Gates 0 to 12 (Build Guide, section 14). The firmware has not yet run on an ESP32.
- The heat model's values are estimates until Gate 9 measures them on the real cells.
- The clamp part is chosen at Gate 1 after a leakage test.

> **Safety rules.** No lithium cell is connected before Gate 3. The first power-up and the first charge are never done alone. A live cell is never driven into thermal runaway, overcharge or over-discharge to prove a limit: limits are proven with simulated signals, the simulator, a heated dummy cylinder, and a supervised 47 Ω resistor. The hardware BMS stays connected as the final backstop. The enclosure is vented and is not a fire enclosure.

13. Likely viva questions, with short answers

Answer in your own words. Short, clear answers are best. If you do not know, say what you do know and where in the project the answer would come from (for example, "Gate 9 measures that").

Basic ideas

1. What is a battery management system (BMS)?

A circuit that watches a battery pack and protects it. It measures cell voltages, current and temperature, and it disconnects the pack when something is unsafe. It can also balance the cells.

2. Why does a lithium-ion pack need a BMS?

Lithium-ion cells are damaged or become unsafe if they are charged too high, discharged too low, carry too much current or get too hot. The BMS keeps every cell inside safe limits.

3. What is thermal runaway?

A chain of heat-making reactions inside a cell that speed up as the cell gets hotter. The heat makes more heat. It can end in fire. It cannot be stopped by disconnecting the pack, so early warning matters.

4. What does 4S1P mean?

Four cells in series (4S) and one cell in each place (1P). The pack has four cells in one row.

5. What are the voltages of your pack?

14.4 V nominal (4 × 3.6 V) and 16.8 V when full (4 × 4.2 V).

6. What is the capacity and energy of the pack?

3.3 Ah, from the cell label. That is about 47.5 Wh at the nominal voltage.

7. Which cell do you use?

The NCR18650GA, an 18650-size NMC lithium-ion cell. 18650 means 18 mm wide and 65 mm long.

8. What is cell balancing?

Keeping all cells at the same charge. The hardware BMS does passive balancing: above 4.18 V it bleeds about 30 mA from a cell that is ahead of the others.

9. What is the current limit of your prototype?

5 A continuous for discharge and 1.5 A for charge. The hardware BMS board is rated 20 A, but 20 A is not the project limit.

10. Why is a fixed temperature limit late?

A cell is large and heats slowly. Its surface needs many minutes to reach 55 °C, even when a fault is already making heat inside. By then the fault has been there a long time.

11. Where does heat in a cell come from?

Mainly from the current flowing through the cell's internal resistance: I^2R, called Joule heat. A fault, like an internal short, adds heat that the current does not explain.

12. What is an internal short?

A small, unwanted path for current inside a cell. It drains the cell even at rest, and it heats the cell. A soft short is a weak one that grows slowly.

The design

13. Why two protection layers?

So that one failure cannot remove all protection. Layer A, the hardware BMS, always works, even if the ESP32 or its program fails. Layer B, the ESP32 supervisor, adds the predictive layer and separate charge and load relays.

14. What does the hardware BMS do?

It stops charging above about 4.25 V per cell, stops discharging below about 2.7 V per cell, protects against short circuit and over-current, and balances the cells.

15. Why are the relays normally open?

Normally open means the contact is open when the relay has no power. If the ESP32 resets, crashes or loses power, both relays open. The safe state is the default.

16. Why are there pull-up resistors on the relay inputs?

The relay inputs are active LOW. The 10 kΩ pull-ups keep the inputs HIGH, so the relays stay off while the ESP32 is starting or if a wire comes loose.

17. Why two relays, one for charge and one for load?

So the supervisor can stop charging but still allow discharge, or the other way round. For example, a high cell voltage opens only the charge relay.

18. Why is there a fuse, and why 7.5 A?

The fuse protects the wiring and the relay path from a large fault current. 7.5 A is above the 5 A working current, so it does not blow in normal use.

19. Why use an ADS1115 for the cell voltages?

It is a 16-bit ADC: much finer and more accurate than the ESP32's own ADC. The leak estimate needs to see changes of a few millivolts.

20. Why a 330 kΩ and 56 kΩ divider?

The top tap is up to 16.8 V, but the ADC input must stay low. The divider makes 16.8 V into about 2.44 V. High values draw very little current from the cells.

21. Why the 10 kΩ resistor and 100 nF capacitor?

Together with the divider they form a low-pass filter. It removes fast noise and passes the slow cell voltage. In the KiCad simulation the cut-off is about 27 Hz.

22. What are the clamps for, and why is the part TBD?

Clamps protect the ADC input from a voltage spike. Their leakage current upsets the high-value divider, so the part is chosen at Gate 1 after a leakage test. The simulation shows that every 10 nA of leakage moves a cell reading by about 4.1 mV.

23. How do you get each cell's voltage?

The ADC measures each tap against the pack's negative end. Cell 1 is the first tap. Each other cell is the difference between two neighbouring taps.

24. Why NTC thermistors?

They are small, cheap and respond quickly. A 10 kΩ NTC at 25 °C changes its resistance a lot with temperature, so the reading is sensitive.

25. Why five NTCs?

One on each of the four cells, and one in free air to measure the ambient temperature. The heat model needs the air temperature.

26. Why only ADC1 pins for the NTCs?

On the ESP32, ADC2 does not work while Wi-Fi is on. So the five NTCs use ADC1 pins, GPIO32 to GPIO36.

27. Why the INA226 with a 5 mΩ shunt?

The INA226 measures current in both directions, charge and discharge, with fine steps of 2.5 µV. The 5 mΩ shunt gives 25 mV at 5 A and wastes only 0.125 W. It replaced the ACS712 Hall sensor in the BOM for a calibrated, bidirectional reading.

28. Why must the INA226 module's R100 resistor be removed?

The module has its own 0.1 Ω shunt. If it stays, it carries part of the current and the reading is about 4.8 % low (KiCad simulation). The 5 mΩ external shunt replaces it.

29. What is Kelvin wiring?

Two separate thin wires that measure the voltage right at the shunt's own pads. The big current does not flow in them, so the wire resistance does not spoil the reading.

30. Why is the LM2596 connected before the shunt?

The LM2596 powers the electronics. If its current went through the shunt, the pack would never look at rest, and the leak estimate could never run.

31. Why the ESP32?

It is low-cost, fast (240 MHz with a floating-point unit), has Wi-Fi for the dashboard, and enough memory. The whole predictive layer uses well under 1 % of its processor.

32. Why is Wi-Fi not part of the safety chain?

Wi-Fi can fail. All protection works with Wi-Fi off. Wi-Fi only carries the dashboard.

33. Why are trips latched?

A latched trip stays on until a deliberate manual reset. A fault must not clear by itself and switch the pack back on. The reset is allowed only when every cell is between 3.20 and 4.10 V, every NTC is below 40 °C, all sensors are plausible, and the cause has been clear for 60 s.

34. How often does the supervisor measure?

Five times a second (5 Hz). Each reading goes through a median-of-5 filter to reject single bad samples.

The predictive layer

35. What makes your BMS predictive?

It runs a heat model of every cell. The model says what the cell should be doing. The difference from the real reading shows heat that should not be there, long before a fixed limit.

36. What is abnormal heat?

Heat that the cell's current cannot explain, measured in watts. For a healthy cell it is near zero, even under load. A fault makes it positive.

37. What is the two-state thermal model?

Each cell has a core and a surface. Heat is made in the core, flows to the surface through a thermal resistance, and from the surface to the air. Each part has a heat capacity.

38. Why estimate the core temperature?

The sensor can only touch the surface. Under load the core is hotter than the surface. The model gives the core temperature, which no sensor can measure.

39. What is a Kalman filter, in simple words?

A method that mixes a prediction from a model with a noisy measurement, trusting each according to how reliable it is. Every sample it predicts, compares with the reading, and corrects. It also estimates things that are not measured, like the core temperature and the abnormal heat.

40. Why not a neural network, like the base paper?

A neural network needs a lot of training data, which does not exist for our cells. A physical model needs no training, every output has a unit, and it is small enough to run on the ESP32.

41. What are the risk levels?

0 Normal, 1 Watch (a notice), 2 Warning (alarm on, the pack keeps working), 3 Trip (both relays open and latch).

42. What abnormal heat causes a Warning or a Trip?

Warning: 0.5 °C/min, about 0.4 W, for 30 s. Trip: 1.0 °C/min, about 0.75 W, for 10 s.

43. How do you convert watts to °C per minute?

Divide the heat by the cell's total heat capacity, about 45 J/°C, and multiply by 60. For example, 0.75 W ÷ 45 J/°C × 60 = 1.0 °C/min.

44. What is the 5-minute check?

Every 10 s the supervisor predicts each cell's temperature 1 and 5 minutes ahead. When the time comes, it compares the prediction with the reading. An error of +1.0 °C or more raises Watch.

45. Why compare a cell with the other three?

All four cells carry the same current, so their abnormal heat should match. A cell out of step with the others is suspicious, even when every value is still small.

46. What is self-acceleration?

When abnormal heat grows as the cell gets hotter, the heat is feeding itself, as at the start of thermal runaway. If, while at Warning, the abnormal heat rises by 50 % while the core rises by 1 °C within 120 s, the supervisor trips at once.

47. What is the leak estimate?

At rest, a cell with an internal short slowly loses voltage compared with the others. The supervisor measures this drift over 60 minutes and turns it into a leak current. 20 mA or more raises Watch.

48. What is the energy cross-check?

A leak current I at voltage V makes heat V × I inside the cell. If this matches the abnormal heat from the thermal model, two independent methods, electrical and thermal, point to the same fault.

49. What is the safe continuous current?

The largest current at which every cell would settle below its temperature limit. The supervisor shows it on the dashboard.

50. How do you detect an NTC that has come off its cell?

Under load, the cell reads cooler than its own current can explain, so the abnormal heat becomes negative. Below −0.4 W for 60 s with at least 1 A is a sensor fault, and both relays open.

51. What happens before the model is fitted?

The thermal parts are switched off and simple rate-of-rise limits are used: 0.5 °C/min Warning and 1.0 °C/min Trip. The model is fitted at Gate 9.

Tools and results

52. Why KiCad?

It is a free, open-source circuit design tool used in industry. It has a built-in circuit simulator, ngspice.

53. What is ngspice?

A SPICE circuit simulator. SPICE programs calculate the voltages and currents in a circuit. KiCad uses ngspice inside.

54. What did the KiCad simulation show?

The ADC inputs stay below 3.0 V; leakage of 10 nA at an ADC input moves a cell reading by about 4.1 mV; the NTC voltage is 1.65 V at 25 °C; the shunt gives 25 mV at 5 A; and the heat model's numbers match Octave's.

55. Why GNU Octave and not MATLAB?

MATLAB needs a paid licence. GNU Octave is free and runs the same .m files, so the work also runs in MATLAB.

56. What did the Octave check show?

Octave's copy of the estimator, given the same data, gives the same core temperature and abnormal heat as the controller program: within 0.007 °C and 0.0013 W on four simulated scenarios.

57. What is Xcos?

The block-diagram simulator in Scilab, a free tool like Simulink. We built the heat model and the abnormal-heat detection as block diagrams.

58. What is the C++ simulator?

A program that runs the real controller code on a PC against a simulated pack: four cells with their heat, the hardware BMS, noisy sensors, and every fault. It has 30 test scenarios and the dashboard demonstrations.

59. How do you know the controller program works?

26 unit tests and 30 scenarios test it in simulation; the Octave copy gives the same numbers; and on the real board each build gate tests it with its own pass rules.

60. What is the main result?

In simulation, with a cell defect under a 3 A load, this project trips about 5 minutes earlier than a fixed-limit BMS, with the cell about 27 °C cooler. A fixed-limit BMS does not react at all to 1 W of heat with no current.

61. Have you tested on real cells?

Not yet. The hardware is the next step, gate by gate. All results now are simulation results. The real values, like the thermal parameters, are measured at the gates.

62. Why not test by heating a real cell until it fails?

That is dangerous and is never done. Limits are proven with simulated signals, the simulator, a heated dummy cylinder, and a supervised 47 Ω resistor that copies a soft short safely.

63. What is new in your project?

In the papers reviewed, none does all of this on a low-cost controller: estimating the abnormal heat in watts with a physical model and Kalman filter, predicting and then confirming each forecast, cross-checking thermal and electrical evidence, and forecasting a safe current.

Limits and future work

64. What happens if the ESP32 crashes?

Both relays open, because they are normally open and their inputs are pulled up. The hardware BMS keeps protecting. The crash is latched.

65. What if a relay contact welds shut?

The hardware BMS and the fuse can still open or clear severe faults. The pack is then removed from service.

66. What if a sensor breaks?

An open, shorted or stuck NTC, or a lost ADS1115 or INA226, is a sensor fault. It latches and opens the relays within about 2 s (60 s for a stuck NTC).

67. What are the limitations?

The heat model's values are estimates until Gate 9 measures them. Heat made inside a cell cannot be stopped by disconnecting; the gain there is warning time. The prototype is limited to 5 A, and charge and load must not be used at the same time.

68. Why can a fixed-limit BMS not see an internal short at rest?

With no current, the pack looks quiet. The heat of a soft short is small, and the temperature rises very slowly. Only an estimate of heat, or of the leak, can see it.

69. What is future work?

Build and test the hardware through Gates 0 to 12, fit the heat model on the real cells, choose the clamp part, then move from perfboard to a PCB and test larger packs.

70. What does the project cost?

About ₹9,260 for the full parts list at replacement prices, including the four cells.

14. Glossary

ADC (analog-to-digital converter)
A circuit that turns a voltage into a number.

ADS1115
A 16-bit, 4-channel ADC chip. It measures the four cell taps.

Ah (ampere-hour)
The charge a battery holds. 3.3 Ah gives 3.3 A for one hour.

Ambient temperature
The temperature of the air around the pack.

Balancing
Keeping all cells at the same charge.

BMS (battery management system)
The protection and monitoring circuit of a battery pack.

Buck converter (LM2596)
A circuit that lowers a DC voltage efficiently. Here it makes 5.0 V from the pack.

CC/CV charging
Constant current, then constant voltage: the normal way to charge lithium-ion cells.

Cell
One battery unit. The pack has four.

Core temperature
The temperature deep inside a cell. No sensor can reach it; the model estimates it.

Cross-check
Checking one result with a second, independent method.

Divider (voltage divider)
Two resistors in series that give a known fraction of a voltage.

ERC (electrical rules check)
KiCad's automatic check of a circuit drawing for wiring mistakes.

ESP32
A low-cost microcontroller with Wi-Fi. It runs the supervisor program.

ESP-IDF
Espressif's official set of tools for writing ESP32 programs.

Firmware
The program that runs inside a microcontroller.

Fixed-limit BMS
A normal BMS that acts only when a reading crosses a fixed value, such as 55 °C.

Forecast
A prediction of a future value, here a cell's temperature 1 or 5 minutes ahead.

Fuse
A wire that melts and opens the circuit when the current is too high.

Gate
One stage of the safe build sequence (Gates 0 to 12), each with pass and stop rules.

GNU Octave
A free program that runs MATLAB-style .m files.

GPIO
A general-purpose pin of the microcontroller.

Heat capacity (J/°C)
How much heat raises a body by 1 °C. A cell has about 45 J/°C.

I²C
A two-wire bus (SDA and SCL) that connects chips to the microcontroller.

INA226
A chip that measures current, through the voltage across a shunt, in both directions.

Internal resistance
The small resistance inside a cell. Current through it makes heat.

Internal short
An unwanted current path inside a cell. It drains and heats the cell.

Joule heat (I^2R)
Heat made by a current I through a resistance R.

Kalman filter
A method that combines a model's prediction and noisy readings to estimate values, including ones not measured.

Kelvin connection
Separate sense wires that measure voltage right at a resistor's pads.

KiCad
A free, open-source program for drawing and simulating circuits.

Latch
A trip that stays until a deliberate reset.

Leak current
The current an internal short draws from its own cell.

Li-ion (lithium-ion)
The rechargeable cell type used here.

Low-pass filter
A circuit that passes slow signals and removes fast noise.

MATLAB
A paid program for engineering calculations. Octave runs its files.

Microcontroller
A small computer on one chip that controls a device.

NMC
Nickel manganese cobalt: the chemistry of the cells used.

Normally open (NO)
A relay contact that is open when the relay has no power.

NTC thermistor
A resistor whose resistance falls as its temperature rises.

ngspice
The circuit simulator inside KiCad.

Observer
A model that runs beside the real system and corrects itself with measurements, to estimate hidden values.

OCV (open-circuit voltage)
A cell's voltage when no current flows and it has rested.

Perfboard
A board with a grid of holes for building circuits by hand, without a PCB.

Pull-up resistor
A resistor that holds a pin HIGH when nothing drives it.

Relay
A switch moved by an electromagnet, controlled by a small signal.

Risk level
The supervisor's verdict: Normal, Watch, Warning or Trip.

RMS error
A kind of average size of errors. Large errors count more.

Scenario
One complete simulated test with its own checks.

Scilab and Xcos
A free engineering program; Xcos is its block-diagram simulator, like Simulink.

Self-discharge
A cell slowly losing charge on its own.

Shunt
A small, precise resistor in the current path. Its voltage shows the current.

Simulation
Calculating how a system behaves, on a computer, instead of building it.

SOC (state of charge)
How full a cell is, in percent.

SPICE
The standard method and program family for simulating circuits.

Surface temperature
The temperature of the cell's outside can, where the NTC sits.

Thermal resistance (°C/W)
How many degrees of difference it takes to push 1 W of heat through a path.

Thermal runaway
Heat-making reactions in a cell that speed up as it gets hotter.

Time constant
How fast something settles. After one time constant it has gone about 63 % of the way.

Trip
The supervisor opening the relays because of a fault.

Watchdog
A timer that resets the controller if its program stops running.